

gem5 RISC-V Multi-Core Performance Analysis: Cache Hierarchy, Microarchitecture, and Multi-Program Contention

Amit Chaudhari

M.Tech, Electrical Engineering, IIT Bombay (2025)

Independent Research Project, May 2026

amit37578@gmail.com

Abstract—This report presents a systematic performance characterization of RISC-V multi-core systems using the gem5 architectural simulator (v25.1). Three representative workloads — matrix multiplication (compute-bound), streaming memory copy (memory-bound), and 2D convolution (AI inference proxy, mixed) — are evaluated across four L1D cache sizes (8–64 kB), two CPU models (in-order TimingSimpleCPU and out-of-order O3CPU), and all multi-program combinations on 2–3 cores. Key findings include: (1) a sharp cache threshold effect at 32 kB for MatMul where L1D miss rate falls from 6.2% to 0.05%; (2) a memory wall preventing O3 from providing more than $1.33\times$ speedup for MemCopy despite $4.54\text{--}6.86\times$ gains for compute-bound workloads; and (3) a bandwidth thief effect where memory-bound workloads degrade compute-bound neighbors by up to 11.6% in shared multi-core environments.

Introduction

Modern SoC and server-class processor designs must balance competing demands: maximising instruction throughput, minimising memory latency, and managing shared resource contention across heterogeneous workloads. Understanding the interaction between microarchitecture choices, cache hierarchy sizing, and multi-program interference is essential for architects working on systems ranging from embedded MCUs to AI accelerators.

This study uses gem5 in Syscall Emulation (SE) mode to simulate a parameterised RISC-V system and answer three questions: (1) At what L1D cache size does a given workload transition from memory-bound to cache-resident? (2) How much does out-of-order execution improve IPC, and is that gain uniform across workload types? (3) How do co-running workloads degrade each other’s performance through shared memory bandwidth?

Experimental Setup

Simulation Infrastructure

All experiments use **gem5 v25.1** compiled for the RISC-V ISA. Simulations run in Syscall Emulation (SE) mode, which provides sufficient fidelity for user-space performance characterisation without the overhead of full-system boot. Table 1 summarises the system configuration.

Table 1: Simulated System Configuration

Parameter	Value
ISA	RISC-V (RV64GCV)
CPU Models	TimingSimpleCPU (in-order) O3CPU (out-of-order, 8-wide)
Clock	3 GHz
L1D Cache	8/16/32/64 kB, 4-way, LRU
L1I Cache	32 kB, 4-way, LRU
L2 Cache	256 kB private per core
Memory	DDR4-2400, Single-channel, 2 GiB
Cache policy	Write-back, Write-allocate
Simulator	gem5 v25.1, SE mode
Cross-compiler	riscv64-linux-gnu-gcc 11.4.0

Workloads

Three C benchmarks are compiled statically with `-O2` for RISC-V:

MatMul — 64×64 integer matrix multiplication. Exhibits $O(N^3)$ arithmetic intensity with good spatial locality once the working set fits in cache (≈ 96 kB for three 64×64 `int` matrices). Classified as *compute-bound* at 32 kB and above.

MemCopy — Four iterations of 4 MB streaming `memcpy`. Working set (≈ 8 MB) far exceeds all cache levels, guaranteeing persistent DRAM access. Classified as *memory-bound* at all cache sizes.

Conv2D — Eight 3×3 convolution filters applied to a 32×32 input map. Serves as an AI inference proxy. Working set is small (≈ 16 kB), fitting even in the smallest cache, but exhibits complex access patterns. Classified as *mixed*.

Experiment Dimensions

- **Cache sweep:** L1D $\in \{8, 16, 32, 64\}$ kB, L2 fixed at 256 kB

- **CPU comparison:** TIMING vs. O3 at each cache size
- **Multi-program:** All combinations of 2–3 workloads on separate cores simultaneously (7 experiments, TIMING and O3)

Total: **31 gem5 simulations** yielding IPC, CPI, L1D/L2 miss rates, and simulation ticks.

Results and Analysis

Cache Hierarchy Sweep (TIMING CPU)

Figure 1 shows IPC as a function of L1D cache size for all three workloads under the in-order TIMING CPU.

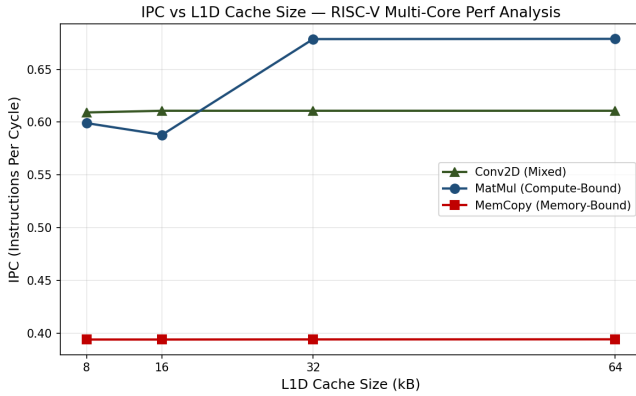


Figure 1: IPC vs. L1D cache size (TIMING CPU). MatMul exhibits a sharp transition at 32 kB; MemCopy is flat; Conv2D is stable throughout.

MatMul shows a pronounced sensitivity to cache size. At 8 kB, the L1D miss rate is 5.32% and IPC is 0.599. At 16 kB, miss rate paradoxically rises to 6.20% (IPC 0.588) due to increased conflict misses in the smaller associativity-constrained cache. At 32 kB, the working set fits entirely and miss rate collapses to 0.051%, lifting IPC to 0.679. No further improvement is observed at 64 kB (IPC 0.679), confirming 32 kB as the optimal point.

MemCopy shows complete insensitivity across all cache sizes — L1D miss rate holds at $\approx 1.34\%$ and IPC remains 0.394 regardless of cache size. The 4 MB working set far exceeds even the 64 kB L1D, ensuring every iteration spills to DRAM.

Conv2D is stable at $\text{IPC} \approx 0.610$ across all cache sizes with miss rate under 0.25%, confirming the working set fits comfortably even at 8 kB.

The L2 miss rate (Figure 2) reveals an inverse pattern for MatMul: as L1D grows and captures more data, L2 receives fewer requests, causing its miss *rate* to rise (from 0.22% at 8 kB to 38.64% at 32 kB and 47.55% at 64 kB).

MemCopy’s L2 miss rate is a steady 77%, confirming persistent DRAM pressure.

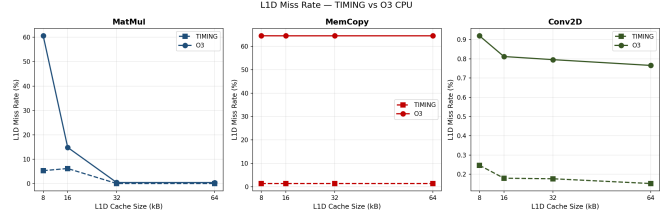


Figure 2: L1D miss rate vs. cache size for TIMING (dashed) and O3 (solid). O3 generates far more misses for MemCopy due to aggressive prefetch behaviour.

In-Order vs. Out-of-Order CPU (O3)

Figure 3 quantifies the IPC speedup of O3 over TIMING at 32 kB L1D. Table 2 provides the full numerical comparison.

Table 2: IPC Comparison: TIMING vs. O3 at 32 kB L1D

Workload	TIMING	O3	Speedup
MatMul	0.679	3.083	4.54×
Conv2D	0.611	4.190	6.86×
MemCopy	0.394	0.524	1.33×

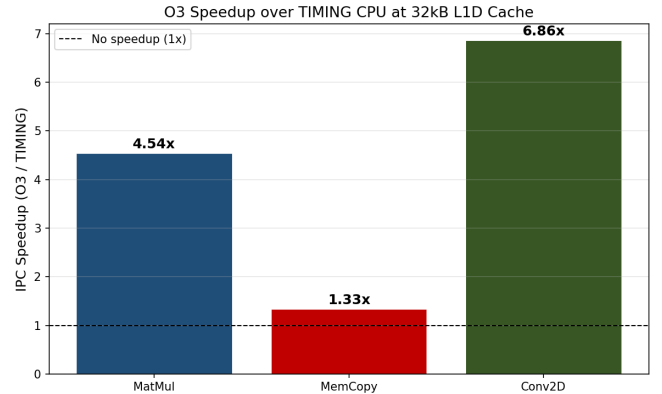


Figure 3: O3 IPC speedup over TIMING at 32 kB L1D. Conv2D benefits most (6.86 $\times$); MemCopy is barely affected (1.33 $\times$).

Conv2D achieves the highest O3 speedup (6.86 $\times$) because its small working set remains entirely cache-resident, allowing the O3 core to exploit instruction-level parallelism (ILP) across independent multiply-accumulate operations without memory stalls. With IPC 4.19, the simulated core is issuing ~ 4 effective instructions per cycle — approaching the theoretical peak.

MatMul’s 4.54 $\times$ speedup reflects similar ILP exploitation, though its larger working set (at 32 kB) introduces oc-

casional L2 misses (66.41%), partially limiting the O3 advantage.

MemCopy’s marginal $1.33\times$ gain illustrates the **Memory Wall**: the O3 core’s ability to issue multiple memory operations simultaneously only aggravates cache pressure. As shown in Figure 2, O3 drives MemCopy’s L1D miss rate from 1.34% to 64.52%, and L2 miss rate reaches 92.64%. The bottleneck has simply moved from instruction throughput to DRAM bandwidth — a fundamental limit no microarchitectural improvement can overcome.

The cache sweep under O3 (Figure 4) reveals that O3 is significantly more cache-sensitive than TIMING for MatMul: at 8 kB, O3 IPC drops to 1.659 (vs. 0.599 for TIMING), because the O3 core generates more concurrent cache requests that cannot be satisfied by a small cache. Above 32 kB, O3 and TIMING both plateau, confirming the cache threshold is architecture-independent for this working set.

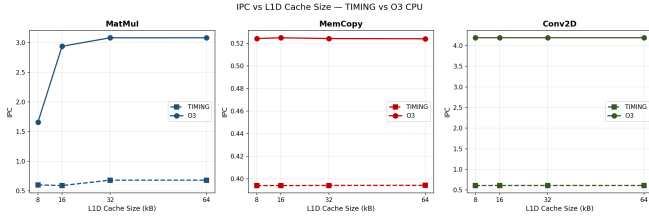


Figure 4: IPC vs. L1D cache size for TIMING (dashed) and O3 (solid). MatMul’s O3 IPC is highly cache-sensitive; Conv2D’s O3 IPC is flat (cache-resident); MemCopy lines are nearly coincident (memory-bound).

Multi-Program Contention

Seven multi-program experiments were run in gem5 SE mode using `set_se_multi_binary_workload`, assigning one workload per core. Figure 5 and Table 3 summarise the IPC degradation relative to solo execution.

Table 3: Multi-Program IPC Degradation vs. Solo (O3, 32 kB)

Workload	Co-runners	Solo IPC	Delta
Conv2D	+ MemCopy	4.190	−9.37%
Conv2D	+ MatMul	4.190	−2.17%
Conv2D	+ MatMul + MemCopy	4.190	−11.63%
MatMul	+ MemCopy	3.083	−3.29%
MatMul	+ Conv + MemCopy	3.083	−4.07%
MemCopy	+ Any	0.524	< 0.1%

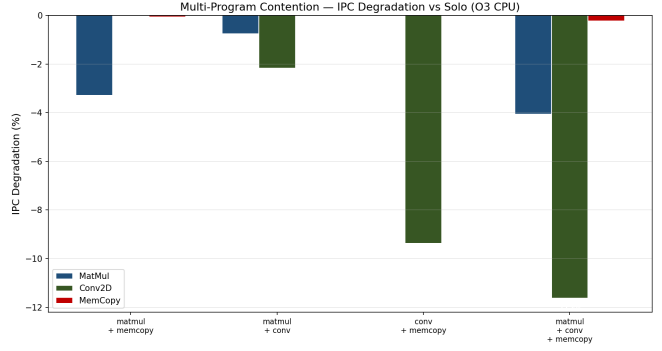


Figure 5: IPC degradation per workload combination (O3 CPU). MemCopy (red) is immune; Conv2D (green) suffers most.

Under TIMING, contention is negligible across all combinations ($< 1.2\%$), as in-order cores generate insufficient concurrent memory traffic to saturate the shared bus.

Under O3, the **Bandwidth Thief Effect** emerges clearly: MemCopy suffers essentially no degradation regardless of its co-runners (it is already bandwidth-saturated), while Conv2D degrades by 9.37% when paired with MemCopy alone and by 11.63% in the 3-core experiment. Figure 6 shows this degradation scaling monotonically with the number of memory-intensive neighbours.

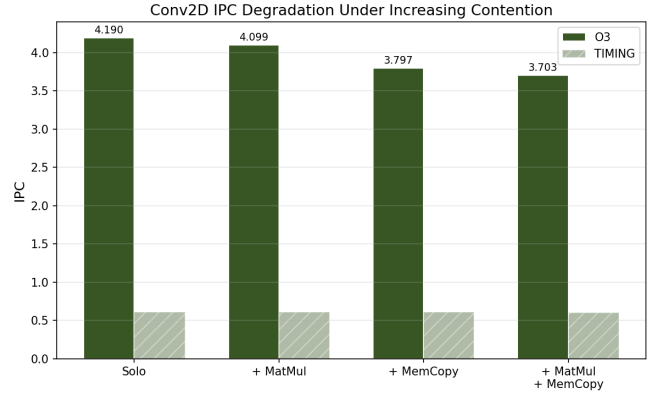


Figure 6: Conv2D IPC degradation under increasing contention (O3). TIMING bars (hatched) are nearly unchanged; O3 bars decline steadily as memory neighbours increase.

Figure 7 provides a direct visual comparison of solo vs. 3-core IPC. Conv2D’s tall bar shortens most visibly, while MemCopy’s short bar is virtually unchanged — confirming it as the bandwidth aggressor.

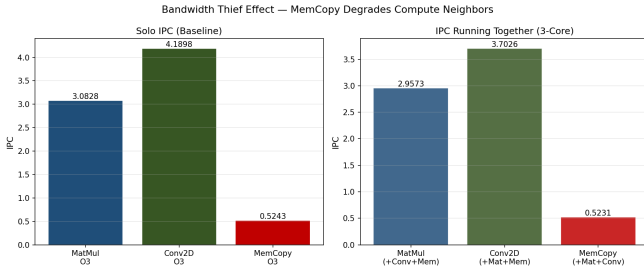


Figure 7: Solo IPC (left) vs. 3-core co-execution IPC (right). MemCopy is the bandwidth thief: unaffected itself, yet degrades compute-bound neighbours.

Key Observations

O1 — Cache threshold effect: MatMul has a sharp working-set threshold at 32 kB. Below this, the L1D miss rate exceeds 5% and IPC degrades significantly regardless of CPU model. This motivates careful working-set sizing in MCU and embedded AI system design.

O2 — Memory wall is absolute: Despite O3’s aggressive memory-request issuance, MemCopy’s DRAM bandwidth limit is reached regardless of cache size or CPU sophistication. The L2 miss rate of 92.64% under O3 confirms that deeper caches and better CPUs cannot compensate for fundamentally streaming access patterns.

O3 — O3 amplifies cache pressure: For memory-bound workloads, O3’s wider issue window generates more concurrent misses, driving L1D miss rate from 1.34% (TIMING) to 64.52% (O3). This is a counter-intuitive result: a “better” CPU makes cache utilisation worse for streaming workloads.

O4 — Contention is asymmetric: In a shared multi-core system, memory-bound workloads act as bandwidth thieves. They saturate DRAM bandwidth with no performance benefit to themselves, while silently degrading compute-bound neighbours by up to 11.6%. This asymmetry has direct implications for workload scheduling on multi-core SoCs.

O5 — TIMING hides contention: Simple in-order CPU models show near-zero contention ($< 1.2\%$) even under 3-core co-execution. Contention analysis using only TIMING models would fail to predict real-world interference — validating the necessity of O3-level simulation for multi-program studies.

O6 — Conv2D models DNN workload behaviour: Conv2D’s high O3 IPC (4.19, near-peak compute utilisation) and sensitivity to memory neighbours mirrors behaviour observed in real DNN inference accelerators. Its degradation pattern under contention is directly relevant to systems running mixed ML and data-movement tasks simultaneously.

Conclusion

This study demonstrates that gem5-based architectural simulation provides actionable insights for RISC-V multi-core system design. The three core findings — cache threshold effects, the memory wall, and the bandwidth thief effect — each correspond to real design decisions: L1D sizing, memory subsystem bandwidth provisioning, and workload scheduling policy.

Notably, the study confirms that simple in-order CPU models are insufficient for contention analysis, and that out-of-order execution fundamentally changes the memory pressure profile of bandwidth-bound workloads. These results have direct relevance to the design of MCU architectures, AI accelerators, and multi-tenant SoC platforms where mixed workload interference determines system-level performance.

Future work includes extending the analysis to OpenMP-parallelised workloads in FS mode, sweeping L2 cache size and associativity, and incorporating hardware prefetcher models to evaluate prefetch effectiveness across workload types.

Experimental Artefacts

All simulation configurations, workload sources, parsing scripts, and raw results are available at: <https://github.com/AmitDragon-India/gem5-riscv-perf>

Tools: gem5 v25.1 · riscv64-linux-gnu-gcc 11.4.0 · Python 3.12 · matplotlib 3.10 · Ubuntu 24.04 WSL2